

Ağaç Algoritmaları

Veri Yapıları ve Algoritmalar

İçerik

- Huffman Kodlaması
- Karar Ağaçları
- Heap

Huffman Ağacı ve Kodlaması

- N sembolden oluşan bir alfabe ile yazılmış bir metin:
 - Her sembole bir dizi bitten oluşan bir kod kelimesi (codeword) verilerek şifrelenebilir.
 - Her sembol için m ($m \geq \log_2 n$) uzunluğunda sabit bir kod verilebilir (fixed-length encoding) (Ör: ASCII Code Sistemi)
 - Daha kısa bit stringleri üretmenin bir yolu sık kullanılan sembollere kısa, az kullanılan sembollere uzun bit stringleri vermek olabilir. Bu yöntemle değişken uzunlukta kodlama ismi verilir (Variable-length encoding).
 - Ör: Morse Alfabesi. Sık kullanılan e: (.) ve a: (.-),
 - az kullanılan q: (---.) ve z: (---..)

Huffman Ağacı ve Kodlaması

- Değişken uzunlukta kodlama yöntemindeki problem:
 - Kodlanmış bir metnin i. sembolünü gösteren bit sayısı kaçtır? Veya hangi bitten başlar?
 - Bu karmaşıklığın önüne geçmek için önek kodları (prefix code) kullanılabilir
 - Önek kodunda, bir sembole ait kod kelimesi başka bir sembolün öneki olamaz
 - Böylece ilk gelen bit grubunda bir sembolün öneki bulunduğunda önekle sembol yer değiştirilerek kod çözme işlemi başlar. Bu işle bit dizisi sonlanana kadar devam eder

Huffman Ağacı ve Kodlaması

- Bir alfabe için binary önek kodlaması yapılmak isteniyorsa:
 - Alfabenin sembolleri ikili ağacın yapraklarına yerleştirilir
 - İkili ağacın sol ayrıtları 0 ile, sağ ayrıtları 1 ile etiketlenir
 - Bir harfin kodu kökten o sembolün bulunduğu yaprağa kadar olan yolun etiketleri ile ifade edilir
 - Bir yaprak üzerinden başka bir yaprağa devam eden bir yol olamayacağı için hiçbir sembolün kodu başka bir sembolün öneki olamaz

Huffman Ağacı ve Kodlaması

- Sembollerinin rastlanma sıklıkları bilinen n sembollü alfabe için bir ağaç oluşturma amacıyla Huffman tarafından bir açgözlü algoritma geliştirilmiştir.
 - N adet bir düğümlü ağaç oluştur ve alfabenin sembolleri ile etiketle. Her sembolün sıklığını (frekans) ağacın köküne ağırlık olarak kaydet.
- (Bir ağacın ağırlığı = Yapraklarının ağırlık toplamı)
 - En düşük ağırlıklı iki ağacı bul.
 - Onları yeni bir ağacın sol ve sağ alt ağaçları yap.
 - Yeni ağacın ağırlığını köke kaydet.
 - Bu işleme tek bir ağaç oluşana kadar devam et.
- Bu ağaca Huffman ağacı denir.
 - Huffman ağacı Huffman kodu üretir.

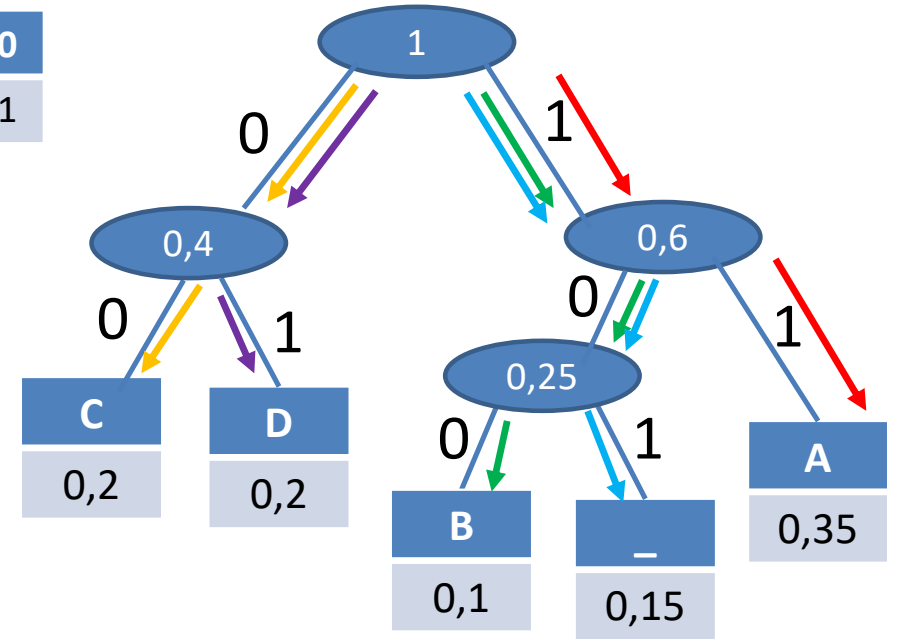
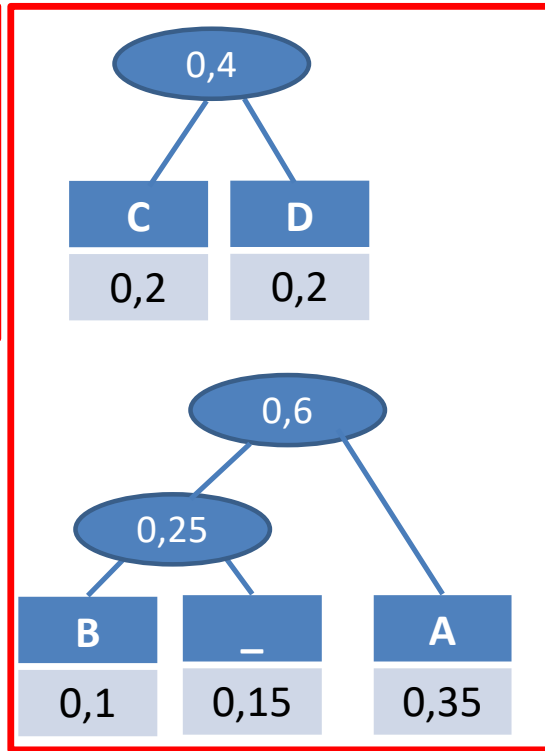
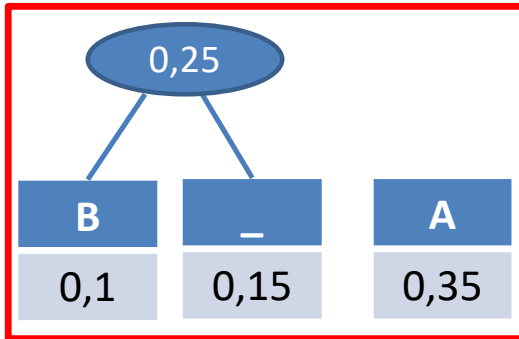
Sembol	A	B	C	D	_
Frekans	0,35	0,1	0,2	0,2	0,15

Sol Ayırıt:	0
Sağ Ayırıt:	1

B	_
0,1	0,15

C	D
0,2	0,2

A
0,35



Sembol	A	B	C	D	_
Kod:	11	100	00	01	101

DAD :	011101
10011011011101 :	BAD_AD

Sembol	A	B	C	D	_
Frekans	0,35	0,1	0,2	0,2	0,15

Huffman Ağacı ve Kodlaması

Sembol	A	B	C	D	_
Kod:	11	100	00	01	101

- Örnek incelendiğinde şifre uzunlukları ve frekans değerleri dikkate alındığında bir sembol için ortalama bit uzunluğu şöyle hesaplanır:

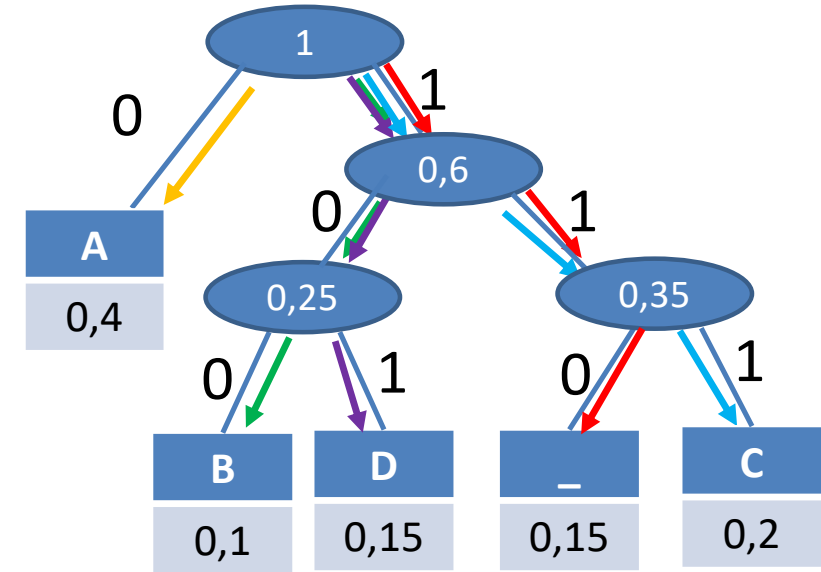
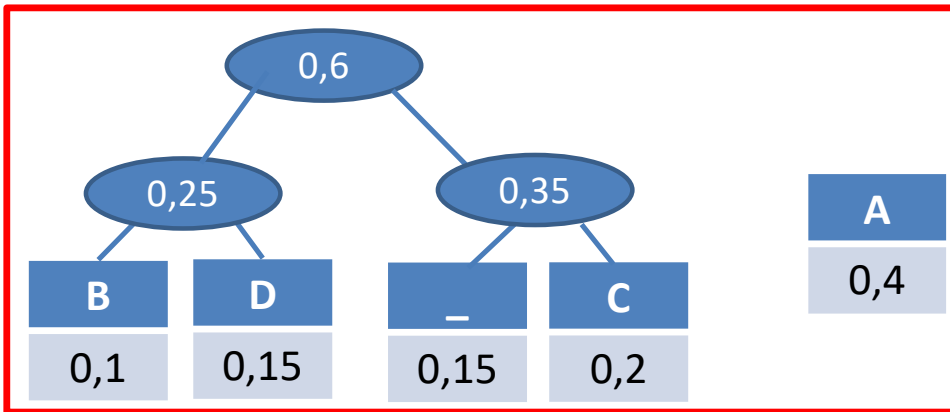
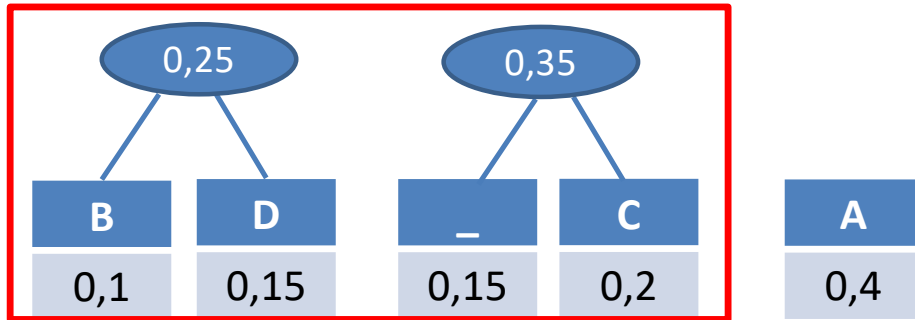
$$2 \cdot 0.35 + 3 \cdot 0.1 + 2 \cdot 0.2 + 2 \cdot 0.2 + 3 \cdot 0.15 = 2.25.$$

- 3 bitlik sabit uzunluklu kodlamaya göre %25 daha az bit kullanıldığı görülmektedir.
 - Bu durumda %25 daha az bellek kullanılacaktır
- Huffman'ın algoritması en önemli veri sıkıştırma algoritmalarından biridir.
- Yapısı gereği kodlanan metnin içerisine kodlama tablosunun da eklenmesi gerekmektedir
 - Bu problem her yeni sembolde kodlama ağacını güncelleyen dinamik Huffman kodlamasıyla çözülebilir
 - Diğer bir algoritma olan Lempel-Ziv ile sadece semboller değil stringler içinde kodlama yapılabilmektedir

Sembol	A	B	C	D	_
Frekans	0,4	0,1	0,2	0,15	0,15

Sol Ayırıt:	0
Sağ Ayırıt:	1

B	D	_	C	A
0,1	0,15	0,15	0,2	0,4



Sembol	A	B	C	D	_
Kod:	0	100	111	101	110

ABACABAD :	0100011101000101
100010111001010:	BAD_ADA

Huffman Ağacı ve Kodlaması

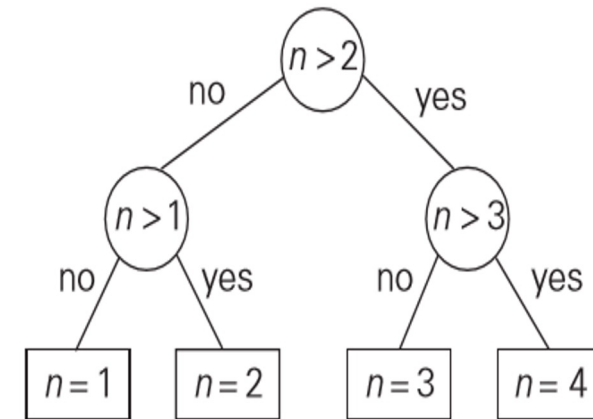
- Huffman algoritması sadece veri sıkıştırma uygulamalarında kullanılmaz
 - N adet pozitif tam sayı ($w_1, w_2, \dots, w_n$) bir ağacın yapraklarına yerleşecekse ağırlıklı yol uzunluğu
$$\sum_{i=1}^n l_i w_i$$
 - Toplamıyla bulunabilir.
 - l_i kökten i. yaprağa kadar olan basit yol uzunluğu
 - Minimum ağırlıklı bir ikili ağaç nasıl oluşturulabilir
 - Bu problem ile genellikle karar verme mekanizmalarında karşılaşılır
 - Karar Ağaçları

Huffman Ağacı ve Kodlaması

- n olasılık içerisinde seçilen bir nesneyi tespit etme problemi
 - Ör: 1 ile n arası tutulan sayıyı tahmin etme oyunu
 - Cevabı Evet/Hayır olan sorular sorarak sonuca ulaşılacak
 - Bu oyun için karar ağaçları ile farklı stratejiler oluşturulabilir
 - Kökten bir yaprağa kadar olan basit yol uzunluğu o yaprakta tutulan sayıya ulaşılması için gerekli soru sayısına eşittir

$$\sum_{i=1}^n l_i p_i$$

- l_i kökten i . yaprağa kadar olan basit yol uzunluğu
- p_i : i sayısının seçilme olasılığı
- Her sayının seçilme olasılığı eşit ise ($1/n$) bir soruda ihtimallerin yarısını elemek mantıklıdır



Huffman Ağacı ve Kodlaması

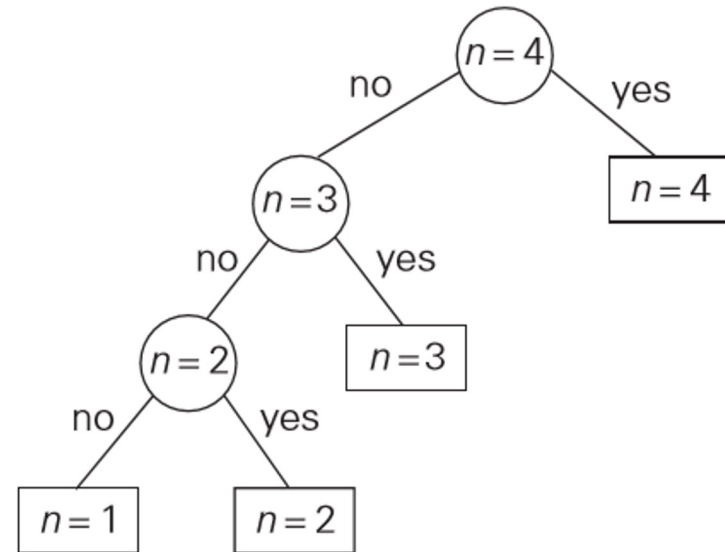
- 1 ile 4 arasındaki sayıların tutulma oranları değişik ise

- $p_1 = 0.1$

- $p_2 = 0.2$

- $p_3 = 0.3$

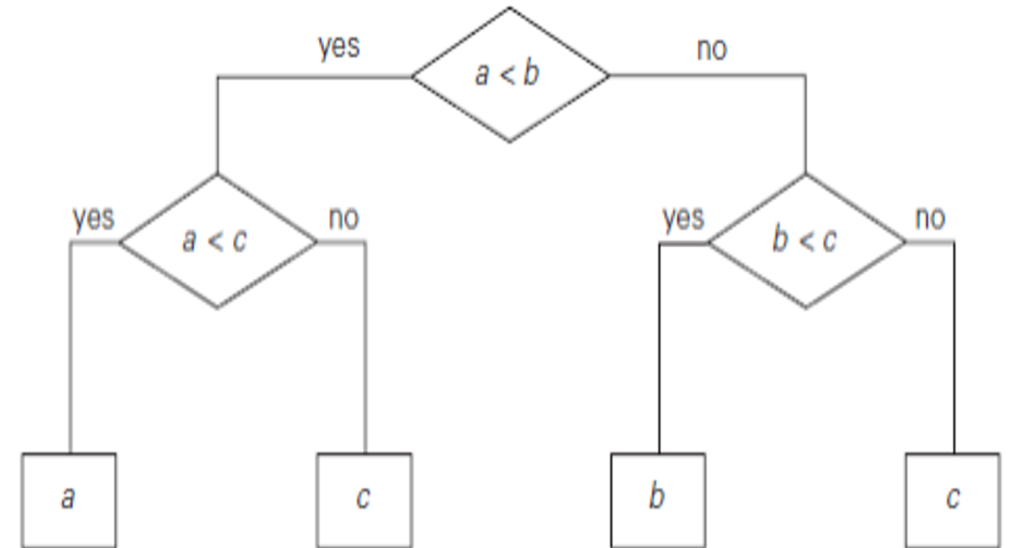
- $p_4 = 0.4$



Karar Ağaçları

- Birçok önemli algoritma girdilerinin öğelerini karşılaştırarak çalışır
 - Sıralama ve arama algoritmaları
 - Bu tür algoritmaların performansı karar ağaçları ile incelenebilir.
 - Her yaprak, algoritmanın n boyutundaki bazı girdilerde çalıştırılmasının olası bir sonucunu temsil eder.
 - Yaprak sayısının sonuç sayısından daha fazla olabilir,
 - Bazı algoritmalar için aynı sonuca farklı bir karşılaştırma zinciri üzerinden ulaşılabilir.

- Birbirinden farklı 3 tam sayının en küçüğünü bulan karar ağacı algoritması

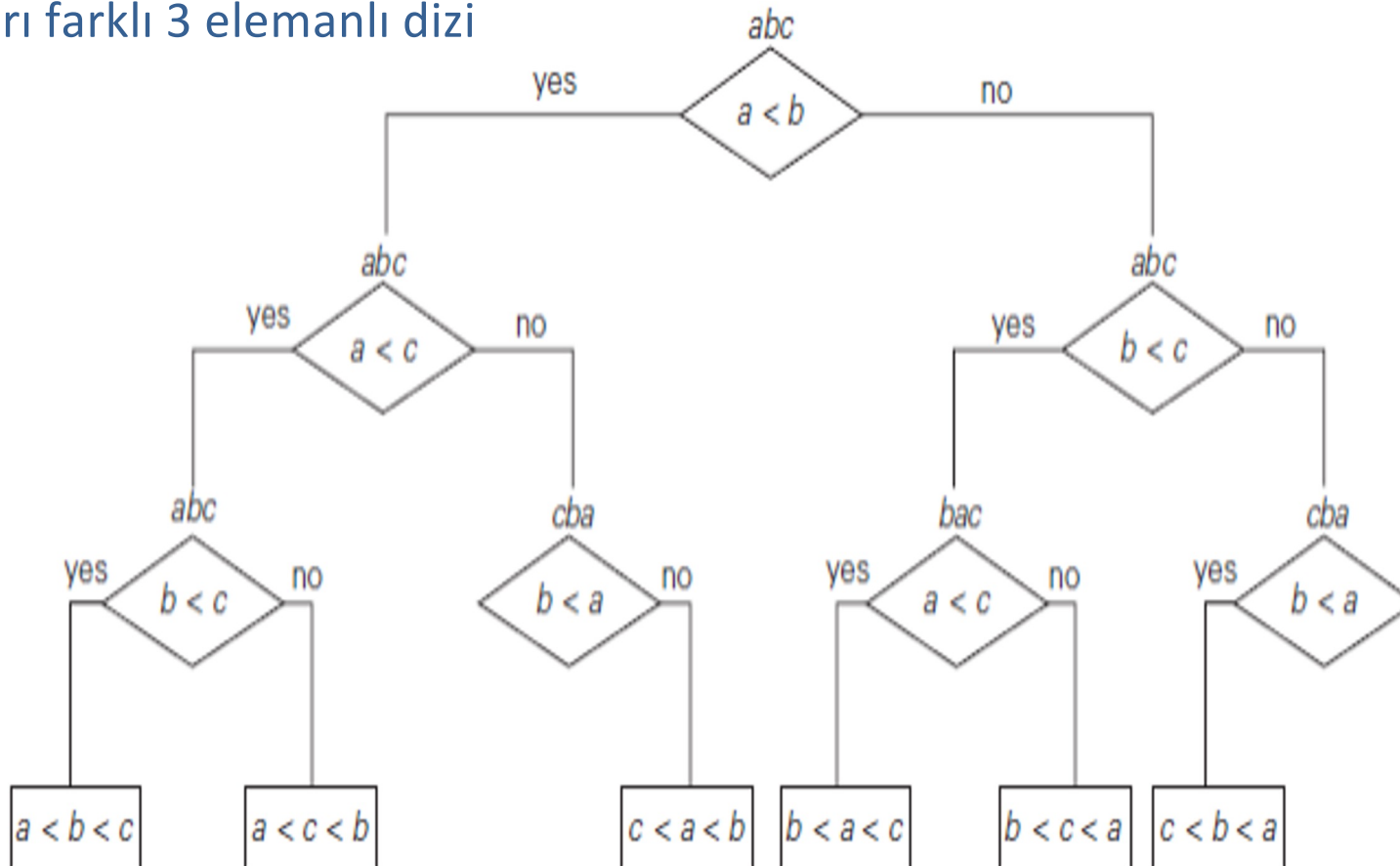


Karar Ağaçları

- Çoğu sıralama algoritması karşılaştırma tabanlıdır,
 - Bu algoritmalar için karar ağaçlarının özelliklerini inceleyerek, zaman verimliliklerinde önemli düşük sınırlar elde edilebilir.
 - Her yaprak bir çözüm çıktısıdır
 - Elemanları bilinmeyen n elemanlı bir dizinin $n!$ şekilde dizilme ihtimali var

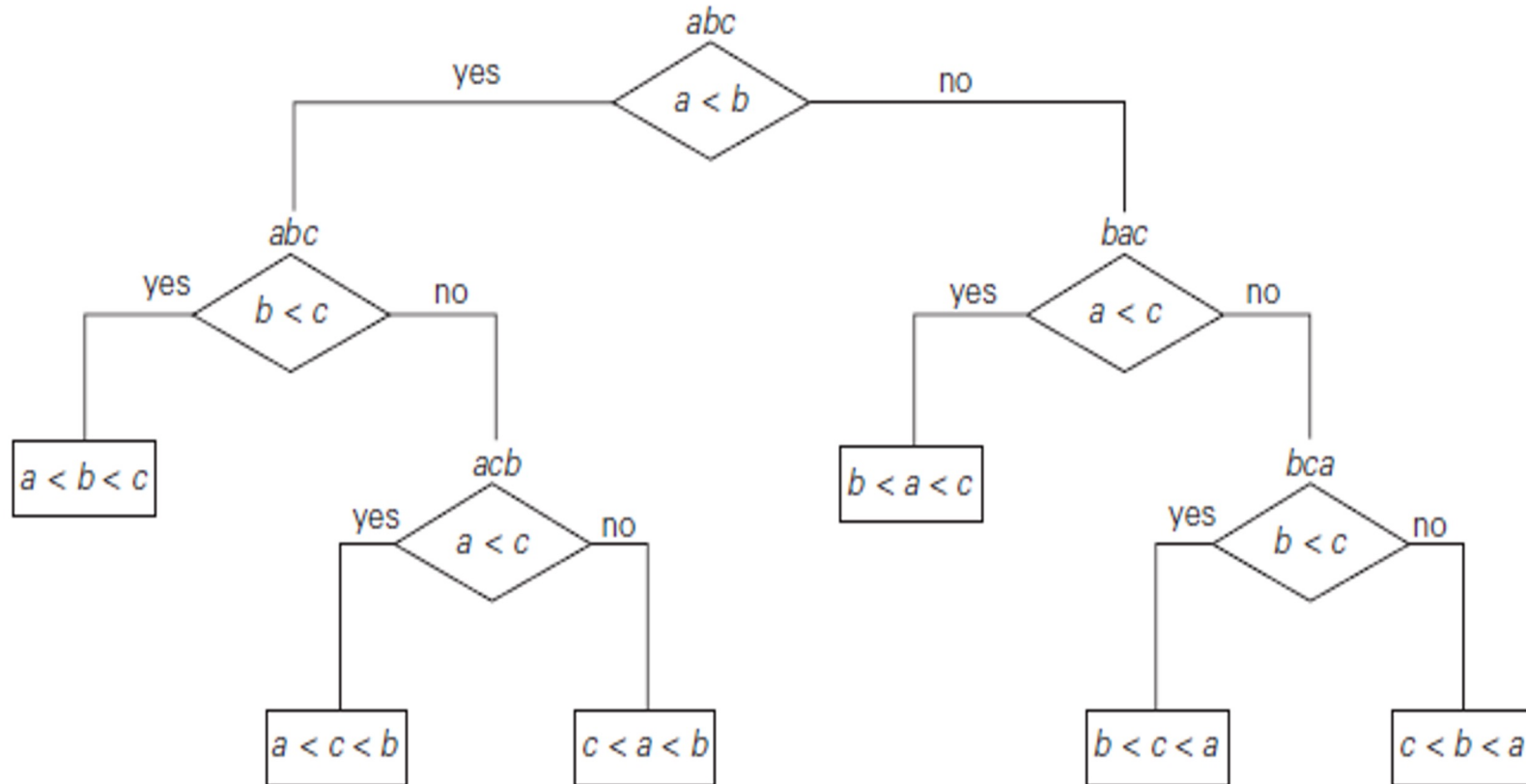
Karar Ağaçları

- Seçimli sıralama için karar ağacı yapısı
 - Elemanları farklı 3 elemanlı dizi



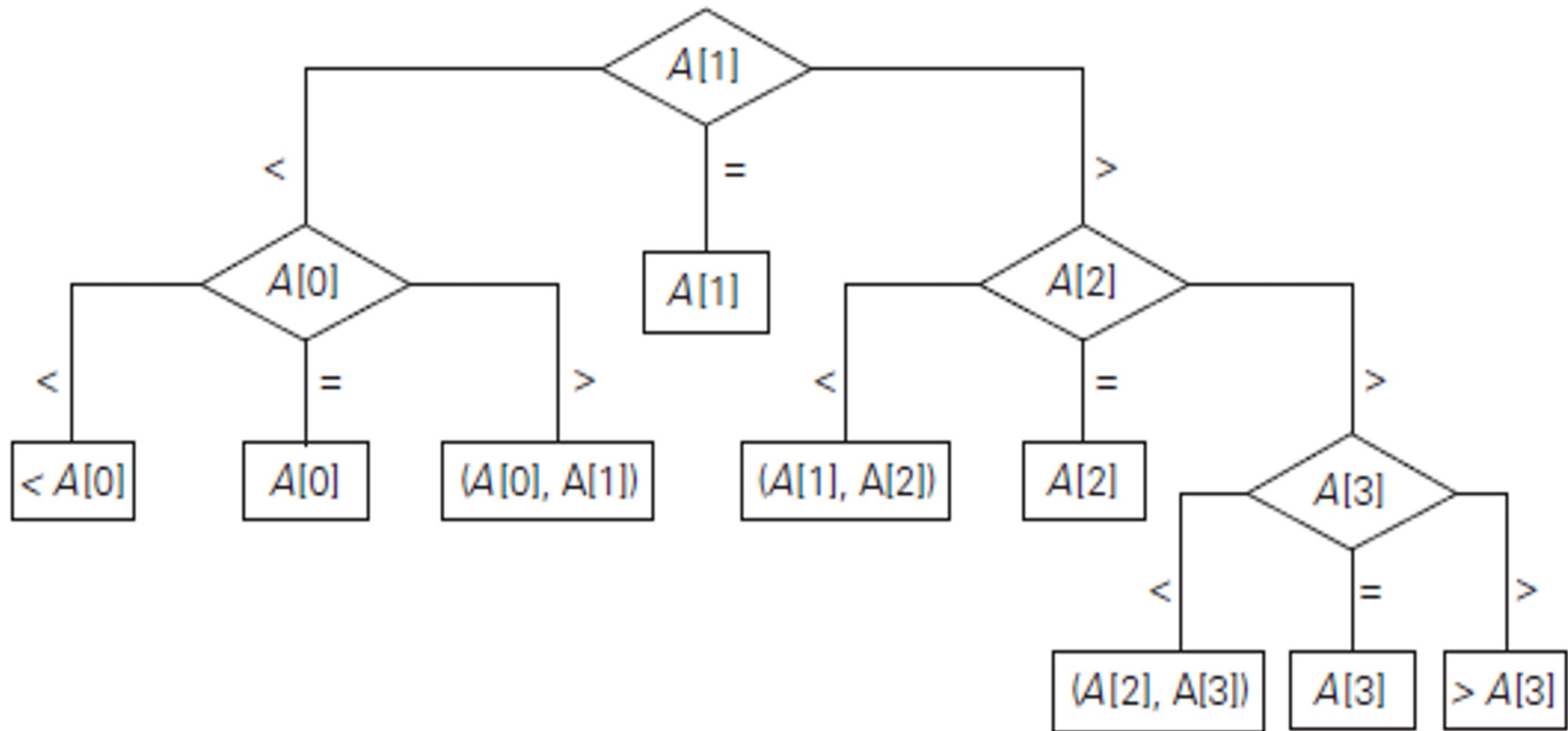
Karar Ağaçları

- Eklemeli sıralama için karar ağacı yapısı



Karar Ağaçları

- Sıralı bir dizide arama için karar ağacı algoritması

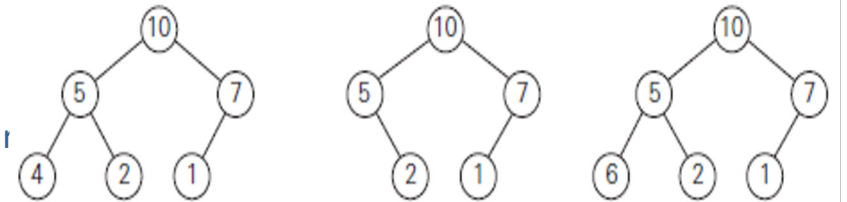


Heap - Öbek - Yığıt

- Heap olarak adlandırılan veri yapısı, kesinlikle düzensiz bir öge yığını değildir.
 - Öncelik sıralarını uygulamak için özellikle uygun olan akıllı, kısmen sıralı bir veri yapısıdır.
 - Bir öncelik sırasının karakteristik bir özelliği olan çok sayıda öğeden oluşur
 - Öncelik sırası işlemleri:
 - En yüksek (yani en büyük) önceliğe sahip bir öge bulmak
 - En yüksek önceliğe sahip bir öğeyi silme
 - Veri setine yeni bir öge ekleme

Heap - Öbek - Yığıt

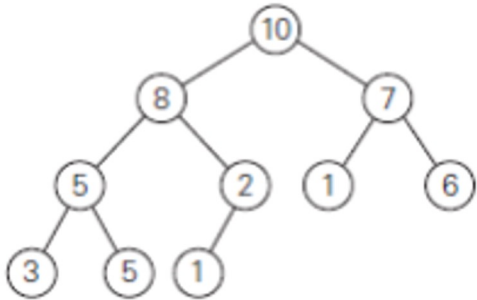
- Heap hem ilginç hem de yararlı kılan, bu operasyonların etkili bir uygulamasıdır.
 - Öncelik kuyrukları,
 - Bilgisayar işletim sistemleri tarafından iş yürütme zamanlaması
 - İletişim ağları tarafından trafik yönetimi.
 - Prim'in algoritması,
 - Dijkstra'nın algoritması
 - Huffman kodlaması gibi bazı önemli algoritmalarda da ortaya çıkarlar.
 - Öbek ayrıca, heapsort adı verilen teorik olarak önemli bir sıralama algoritmasının temel taşı olan veri yapısıdır.
 - TANIM,
 - Aşağıdaki iki koşulun sağlanması şartıyla, düğümlerine atanmış anahtarlar, düğüm başına bir anahtar olan bir ikili ağaç olarak tanımlanabilir:
 - Şekil özelliği
 - Tüm düzeylerde eşit olması gerekir
 - En sağdaki yaprak istisna olabilir
 - Ebeveyn baskınlığı
 - Bir düğümün değeri çocuk düğümlerden büyük veya eşit olmalıdır
 - Bu koşul tüm yapraklar için otomatik olarak gerçekleştirilmeli



Heap Oluřturma

- Heap Kuralları

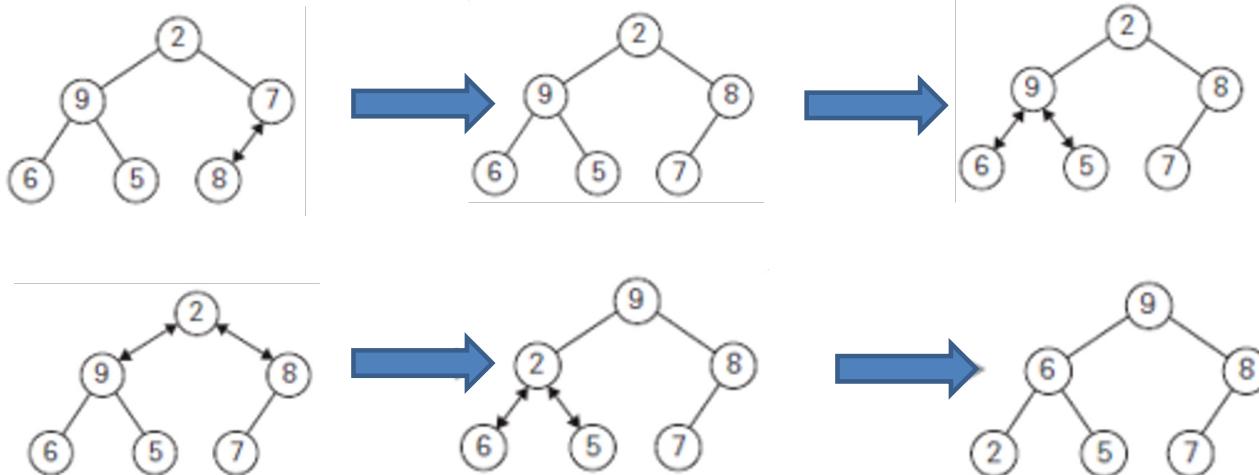
- n adet düğümü olan bir tam ikili ağacın yüksekliğı $\log_2 n$ 'dir
- Heap kökü her zaman en büyük elemanı tutar
- Bir heap düğümünün alt düğümleri de heapdir
- Bir heap dizi şeklinde gösterilecekse;
 - Yukarıdan aşağı ve soldan sağa doğru yerleştirilmeli
 - 0. eleman boş bırakılmalı veya en yüksek değerin yerini göstermeli
 - İlk $n/2$ elemanda ebeveyn, son $n/2$ elemanda çocuk düğüm değerleri olmalı
 - i pozisyonundaki ebeveynin çocukları $2i$ ve $2i+1$ pozisyonlarında olmalı
 - i pozisyonundaki çocuğun ebeveyni $i/2$ pozisyonunda olmalı



0	1	2	3	4	5	6	7	8	9	10
	10	8	7	5	2	1	6	3	5	1
Ebeveynler						Çocuklar				

Heap Oluşturma

- Heap oluşturma yöntemleri
 - Aşağıdan yukarıya oluşturma
 - Dizi elemanları ikili ağaca yerleştirilir
 - Son ebeveynden başlanılarak ebeveyn değeri ile çocuk değeri karşılaştırılır
 - Çocuk değeri büyük ise yer değiştirilir
 - Kök ebeveyne kadar devam eder



ALGORITHM *HeapBottomUp*($H[1..n]$)
//Constructs a heap from elements of a given array
// by the bottom-up algorithm
//Input: An array $H[1..n]$ of orderable items
//Output: A heap $H[1..n]$
for $i \leftarrow \lfloor n/2 \rfloor$ **downto** 1 **do**
 $k \leftarrow i$; $v \leftarrow H[k]$
 $heap \leftarrow false$
 while **not** $heap$ **and** $2 * k \leq n$ **do**
 $j \leftarrow 2 * k$
 if $j < n$ //there are two children
 if $H[j] < H[j + 1]$ $j \leftarrow j + 1$
 if $v \geq H[j]$
 $heap \leftarrow true$
 else $H[k] \leftarrow H[j]$; $k \leftarrow j$
 $H[k] \leftarrow v$

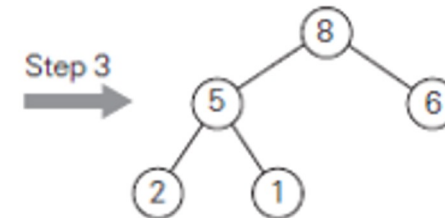
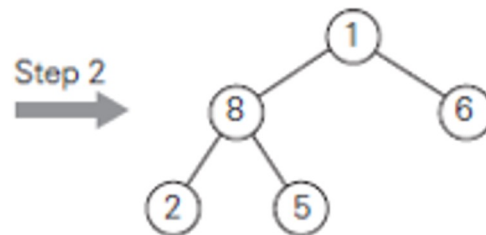
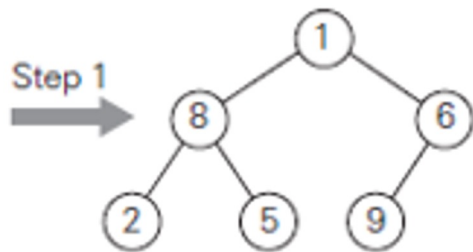
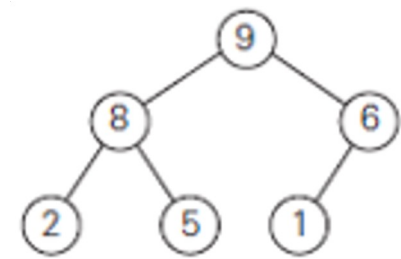
Heap Oluşturma

- Yukarıdan aşağı oluşturma
 - Her elemanın sırayla ağaca eklenmesi
 - Her eklemede heap yeniden düzenlenir



Heap Oluşturma

- Heap'ten eleman silinmesi
 - Silinecek eleman ağacın son elemanı ile yer değiştirir
 - Eleman silinir, ağaç boyutu 1 azaltılır
 - Ağaç tekrar düzenlenir



Heap Sıralama

- İki aşamadan oluşan bir sıralama algoritması
 - Diziden bir heap oluştur
 - Kök düğümü (n-1) kez sil
 - Ters sıralı olarak dizi düzenlenmiş olur

$$O(n) + O(n \log n) = O(n \log n)$$

Stage 1 (heap construction)

2 9 7 6 5 8

2 9 8 6 5 7

2 9 8 6 5 7

9 2 8 6 5 7

9 6 8 2 5 7

Stage 2 (maximum deletions)

9 6 8 2 5 7

7 6 8 2 5 | 9

8 6 7 2 5

5 6 7 2 | 8

7 6 5 2

2 6 5 | 7

6 2 5

5 2 | 6

5 2

2 | 5

2

↓ Sorting the array 2, 9, 7, 6, 5, 8 by heapsort.